

Week 8 – Storing Big Data

02613 Python and High-Performance Computing

“My job was killed because it used too much memory”
- Some of you

“Just request more memory”
- Me

“My job was killed because it used too much memory”

- Student

What if you can't?

Just request more memory”

- Me

Today

Handling data which does not fit into RAM

- Pandas chunking
- NumPy memory mapping
- Zarr arrays

Pandas chunking

Pandas chunking

```
>>> import pandas as pd
>>> df = pd.read_csv('bighuge.csv')
>>> df['coordsx'].sum()
2661914.503899999
```

What if we cannot load the DataFrame?

Pandas chunking

```
>>> import pandas as pd
>>> dfc = pd.read_csv('bighuge.csv', chunksize=100) # Load 100 rows at a time
>>> dfc
<pandas.io.parsers.readers.TextFileReader object at 0x7fd5202f7790>
>>> for df in dfc:
...     print(df.shape, df.columns)
...
<Lots of output>
(100, 7) Index(['coordsx', 'coordsy', 'created', 'observed', 'parameterId',
'stationId', 'value'],
              dtype='object')
(55, 7) Index(['coordsx', 'coordsy', 'created', 'observed', 'parameterId',
'stationId', 'value'],
              dtype='object')
```

Pandas chunking

```
>>> import pandas as pd
>>> dfc = pd.read_csv('bighuge.csv', chunksize=100) # Load 100 rows at a time
>>> dfc
<pandas.io.parsers.readers.TextFileReader object at 0x7fd5202f7790>
>>> coordsum = 0.0
>>> for df in dfc:
...     coordsum += df['coordsx'].sum()
...
>>> coordsum
2661914.5039
```

```
>>> df['coordsx'].sum()
2661914.503899999
```

Pro: don't have to hold full DataFrame in memory.

Con: more complex code

Pandas chunking

```
>>> import pandas as pd
>>> dfc = pd.read_csv('bighuge.csv', chunksize=100) # Load 100 rows at a time
>>> dfc
<pandas.io.parsers.readers.TextFileReader object at 0x7fd5202f7790>
>>> coordsum = 0.0
>>> for df in dfc:
...     coordsum += df['coordsx'].sum()
...
>>> coordsum
2661914.5039
>>> coordsum2 = 0.0
>>> for df in dfc:
...     coordsum2 += df['coordsx'].sum()
...
>>> coordsum2
0.0
```

Can only run through chunks once. Must then re-open reader.

Pandas chunking

```
>>> import pandas as pd
>>> dfc = pd.read_csv('bighuge.csv', chunksize=100) # Load 100 rows at a time
>>> dfc
<pandas.io.parsers.readers.TextFileReader object at 0x7fd5202f7790>
>>> dfc[1]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'TextFileReader' object is not subscriptable
>>> len(dfc)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: object of type 'TextFileReader' has no len()
```

No way to know how long original data is

Pandas chunking

```

1,2020-01-01 00:28:15,2020-01-01 00:33:03,1,1.20,1,N,238,239,1,6,3,0.5,1.47,0,0.3,11.27,2.5
1,2020-01-01 00:35:39,2020-01-01 00:43:04,1,1.20,1,N,239,238,1,7,3,0.5,1.5,0,0.3,12.3,2.5
1,2020-01-01 00:47:41,2020-01-01 00:53:52,1,.60,1,N,238,238,1,6,3,0.5,1,0,0.3,10.8,2.5
1,2020-01-01 00:55:23,2020-01-01 01:00:14,1,.80,1,N,238,151,1,5.5,0.5,0.5,1.36,0,0.3,8.16,0
2,2020-01-01 00:01:58,2020-01-01 00:04:16,1,.00,1,N,193,193,2,3.5,0.5,0.5,0,0,0.3,4.8,0
2,2020-01-01 00:09:44,2020-01-01 00:10:37,1,.03,1,N,7,193,2,2.5,0.5,0.5,0,0,0.3,3.8,0
2,2020-01-01 00:39:25,2020-01-01 00:39:29,1,.00,1,N,193,193,1,2.5,0.5,0.5,0.01,0,0.3,3.81,0
2,2019-12-18 15:27:49,2019-12-18 15:28:59,1,.00,5,N,193,193,1,0.01,0,0,0,0,0.3,2.81,2.5
2,2019-12-18 15:30:35,2019-12-18 15:31:35,4,.00,1,N,193,193,1,2.5,0.5,0.5,0,0,0.3,6.3,2.5
1,2020-01-01 00:29:01,2020-01-01 00:40:28,2,.70,1,N,246,48,1,8,3,0.5,2.35,0,0.3,14.15,2.5
1,2020-01-01 00:55:11,2020-01-01 01:12:03,2,2.40,1,N,246,79,1,12,3,0.5,1.7
1,2020-01-01 00:37:15,2020-01-01 00:51:41,1,.80,1,N,163,161,2,9.5,3,0.5,0,
1,2020-01-01 00:56:27,2020-01-01 01:21:44,1,3.30,1,N,161,144,1,17,3,0.5,4.
2,2020-01-01 00:21:54,2020-01-01 00:27:31,1,1.07,1,N,43,239,1,6,0.5,0.5,1.
2,2020-01-01 00:38:01,2020-01-01 01:15:21,1,7.76,1,N,143,25,1,28.5,0.5,0.5
1,2020-01-01 00:15:35,2020-01-01 00:27:06,3,1.60,1,N,211,234,2,9,3,0.5,0,0
1,2020-01-01 00:41:20,2020-01-01 00:44:22,1,.50,1,Y,234,90,1,4,3,0.5,1,0,0
1,2020-01-01 00:56:38,2020-01-01 01:13:34,1,1.70,1,N,246,142,2,11.5,3,0.5,0,0,0.3,15.3,2.5
2,2020-01-01 00:08:21,2020-01-01 00:25:29,1,8.45,1,N,138,216,2,24.5,0.5,0.5,0,0,0.3,25.8,0
1,2020-01-01 00:25:39,2020-01-01 00:27:05,1,.00,1,N,170,162,4,3,3,0.5,0,0,0.3,6.8,2.5

```

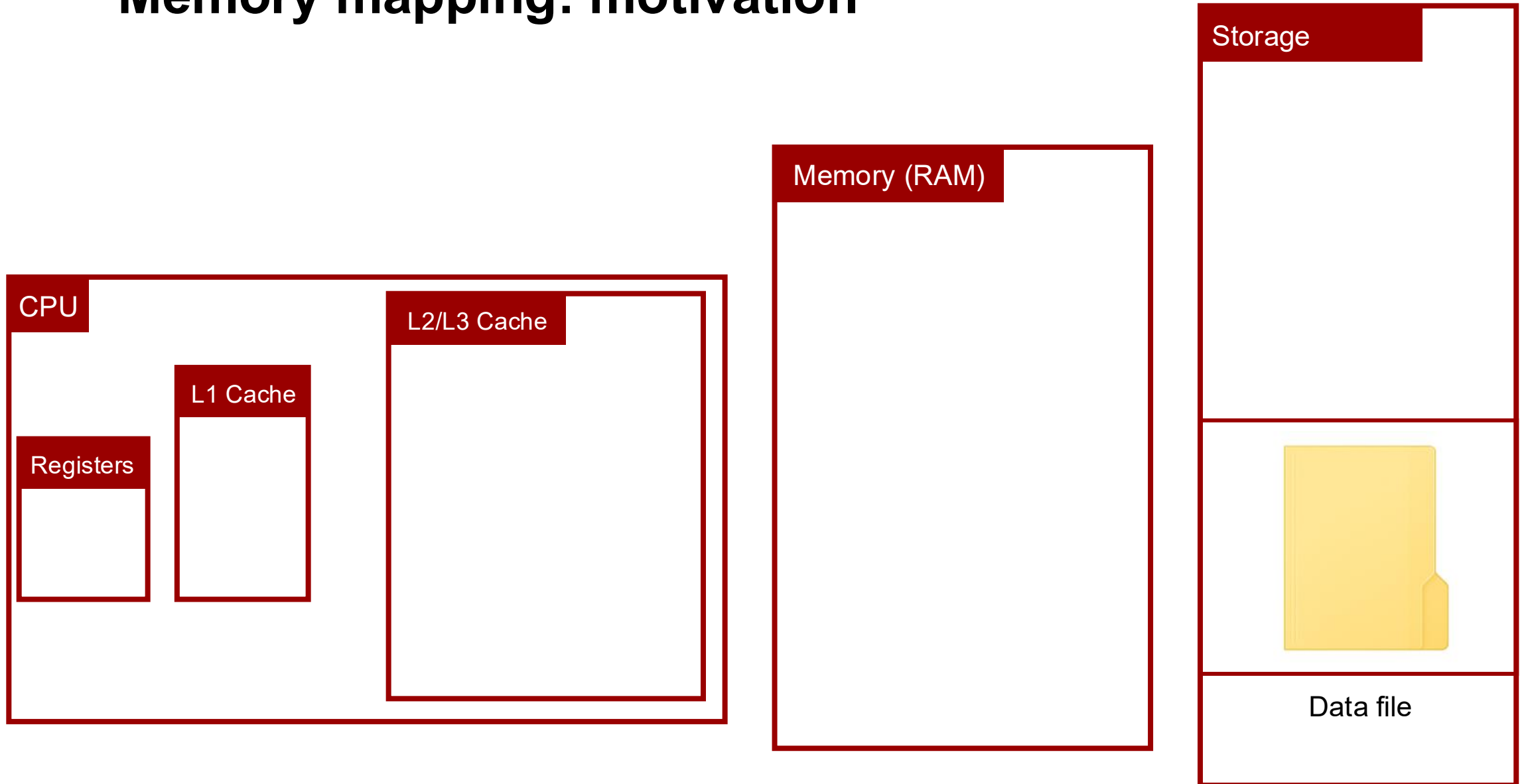
No way to know how long original data is, because each line is different

Parquet chunking

```
>>> import pyarrow.parquet as pq
>>> pf = pq.ParquetFile("bighuge.parquet")
>>> pf.num_row_groups
82
>>> g = pf.read_row_group(2)  # Read 3rd row group
>>> g.shape
(100, 7)
>>> df = g.to_pandas()  # Back to pandas
```

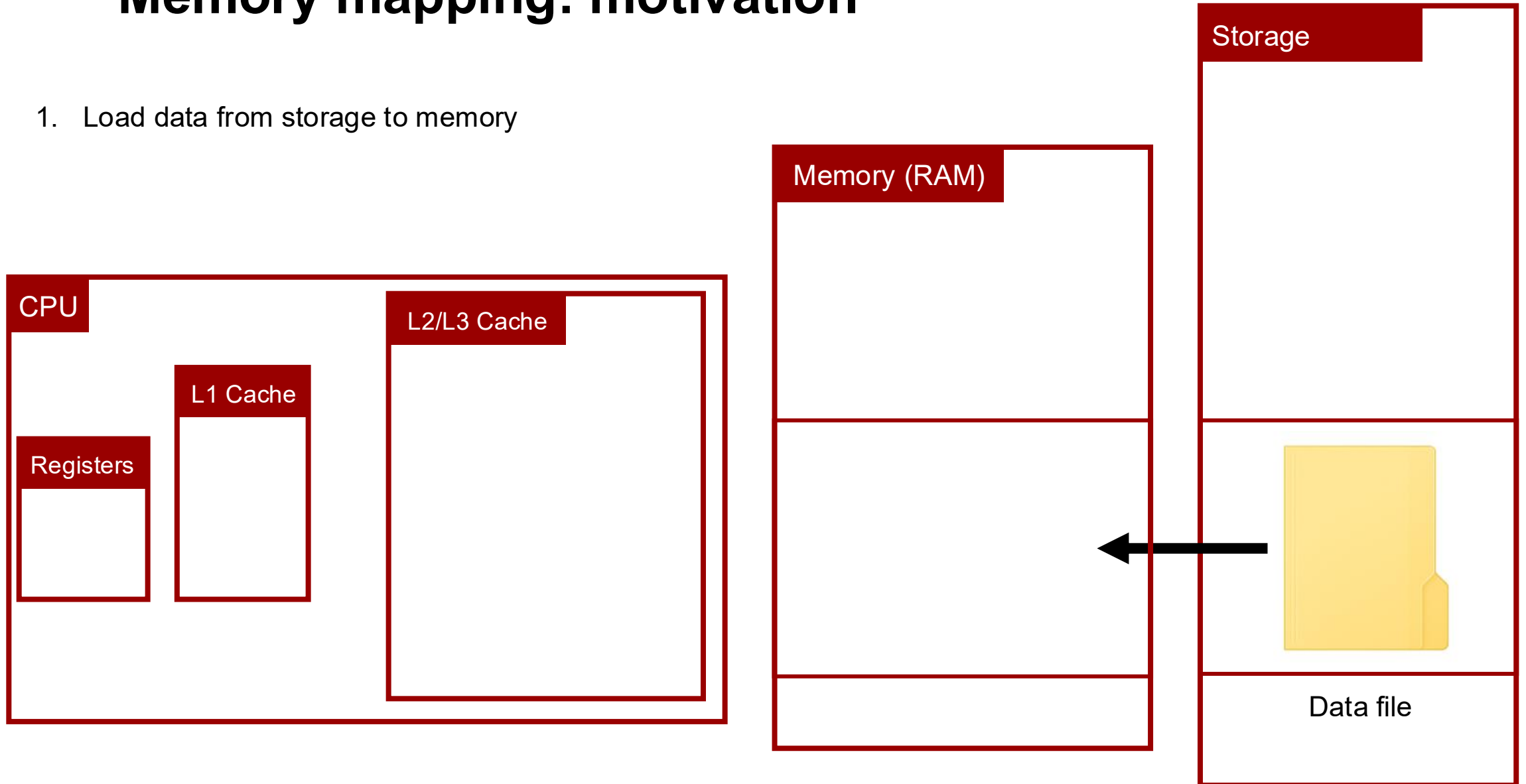
Memory mapping

Memory mapping: motivation



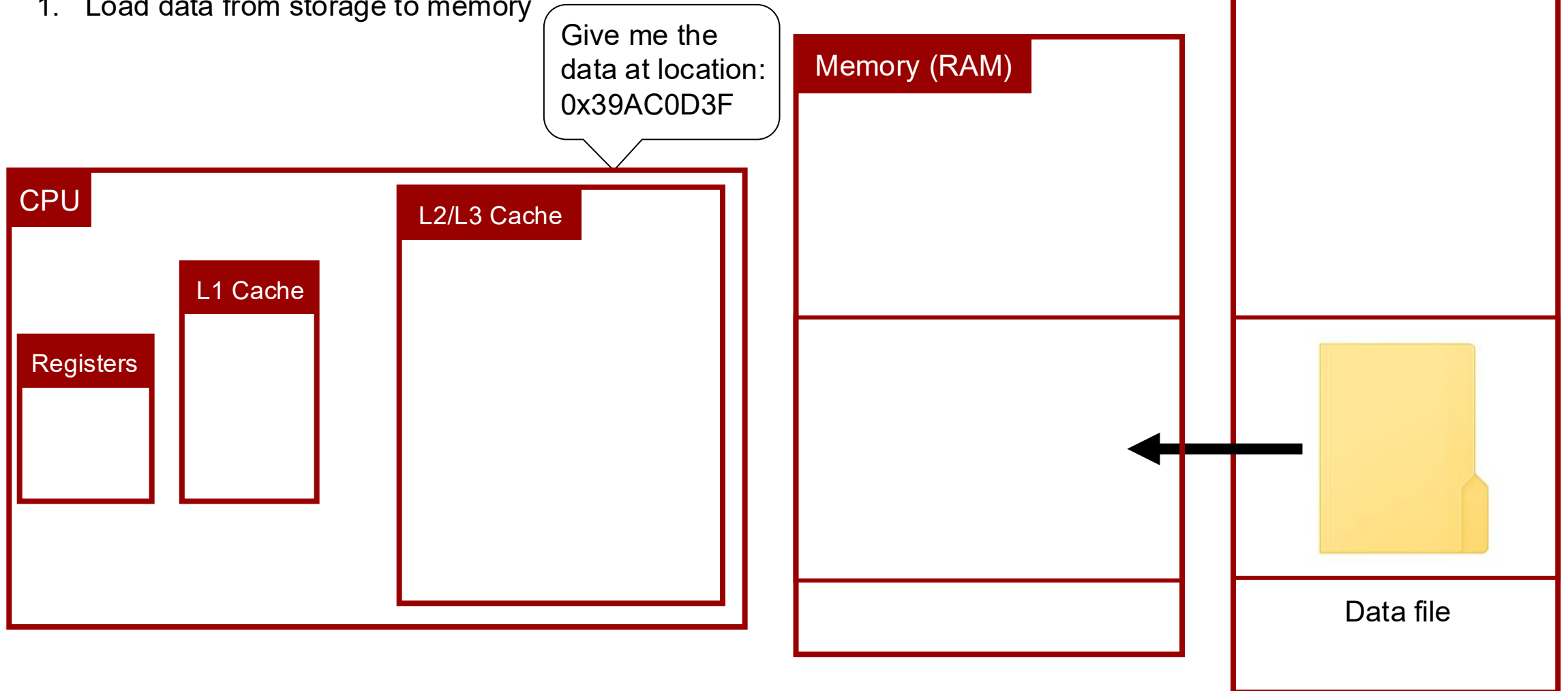
Memory mapping: motivation

1. Load data from storage to memory



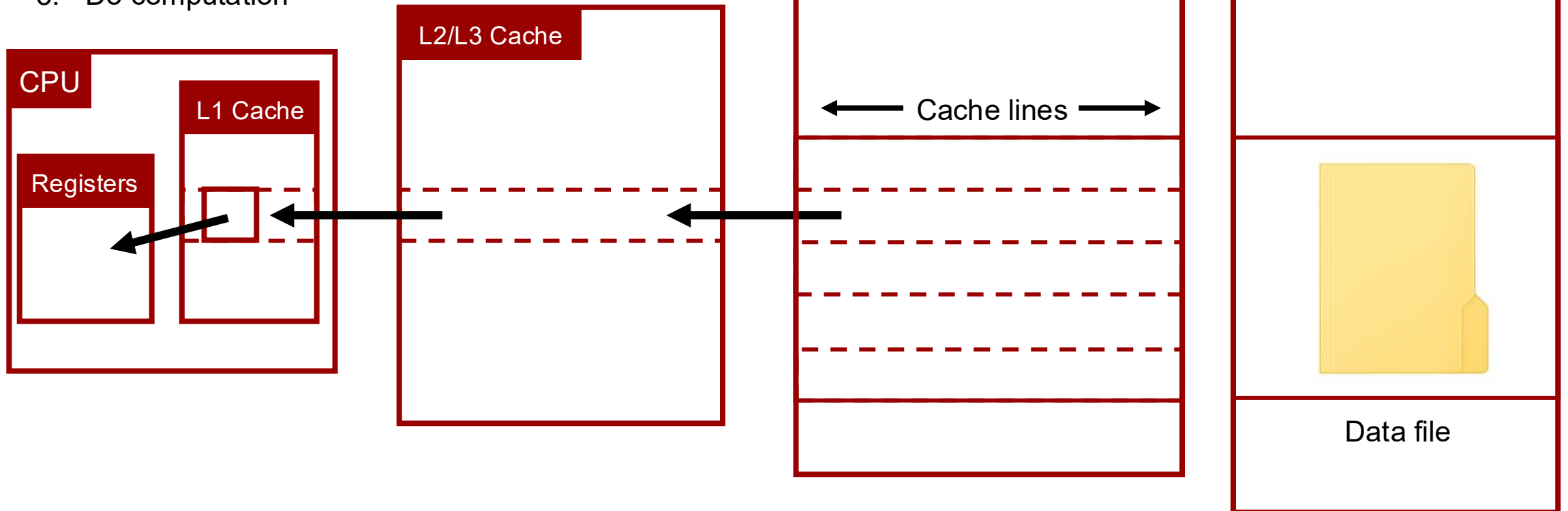
Memory mapping: motivation

1. Load data from storage to memory



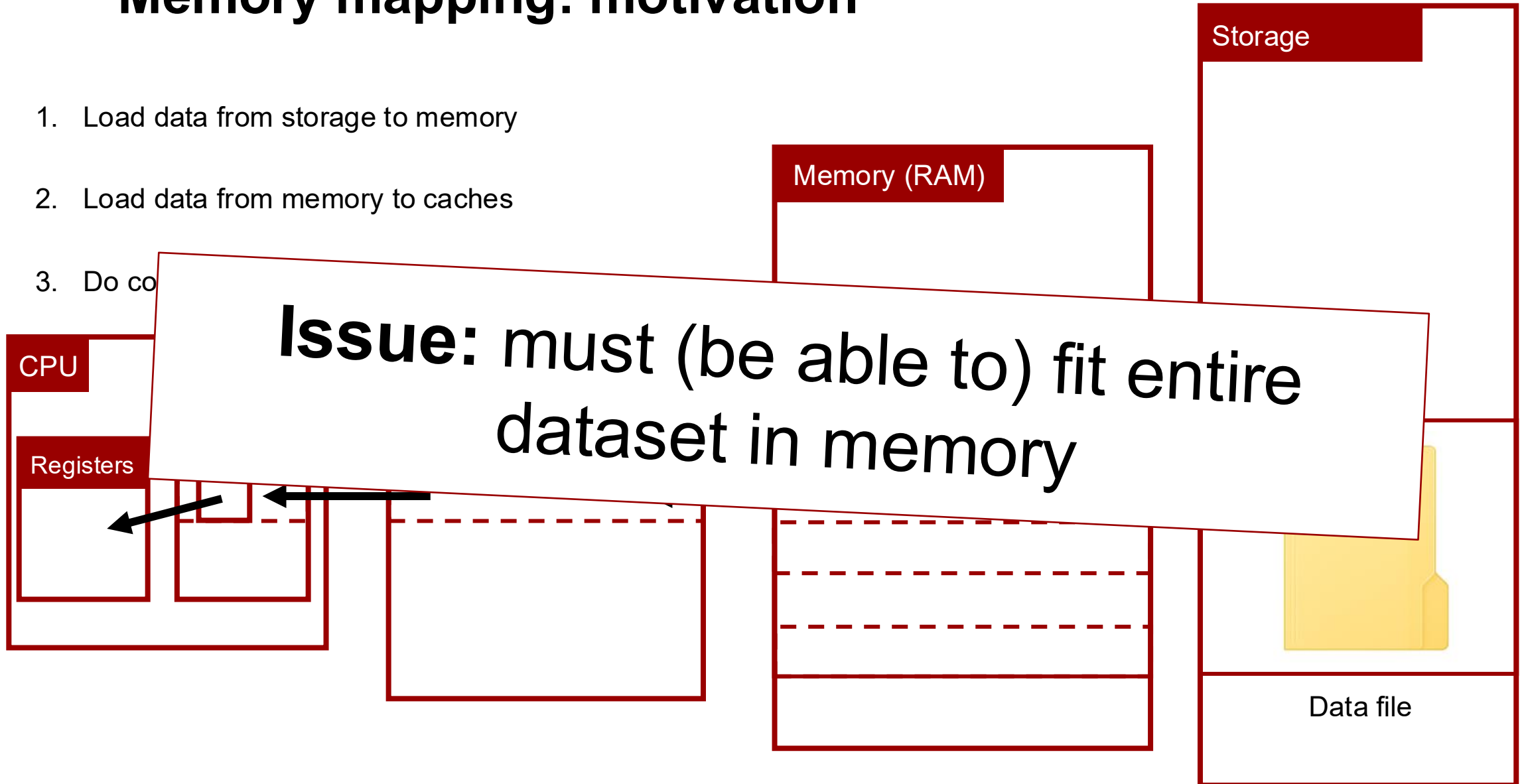
Memory mapping: motivation

1. Load data from storage to memory
2. Load data from memory to caches
3. Do computation

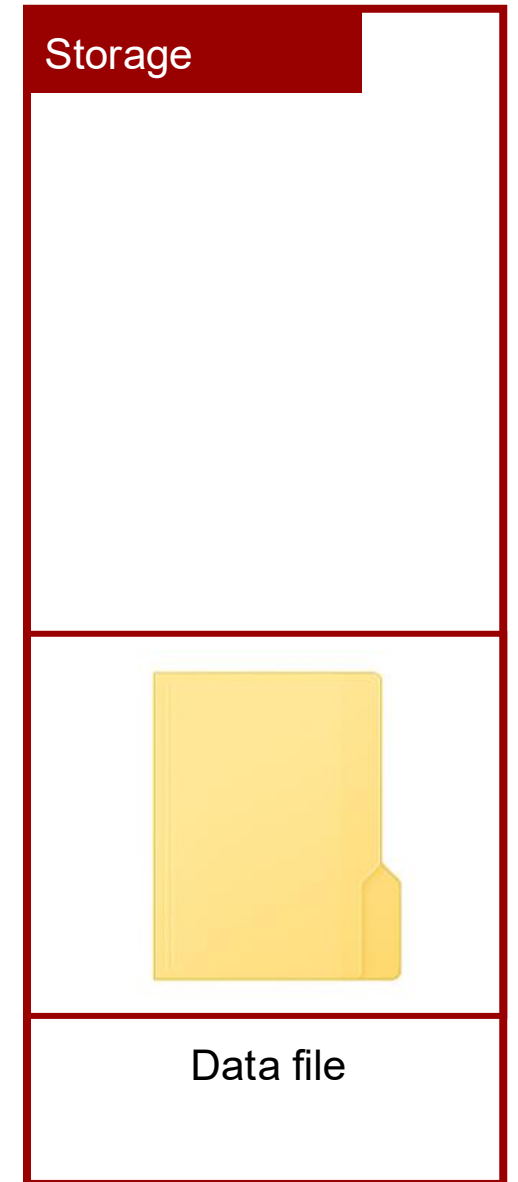
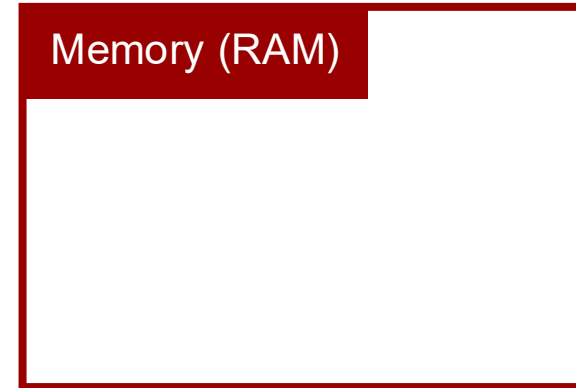
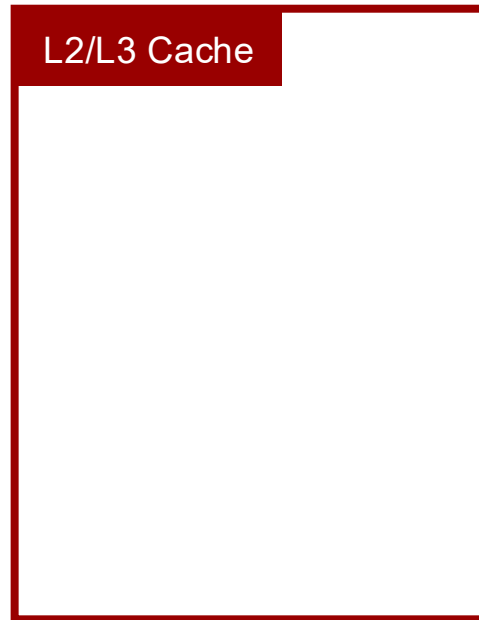
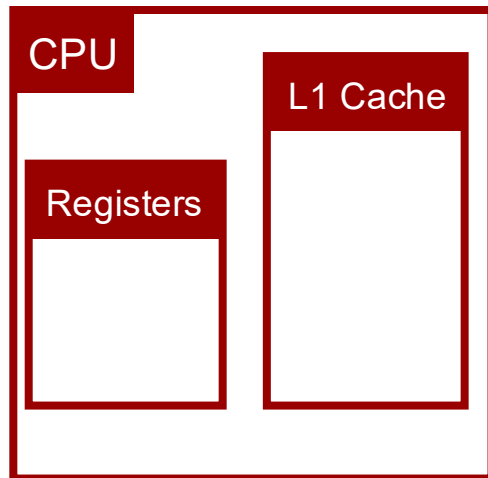


Memory mapping: motivation

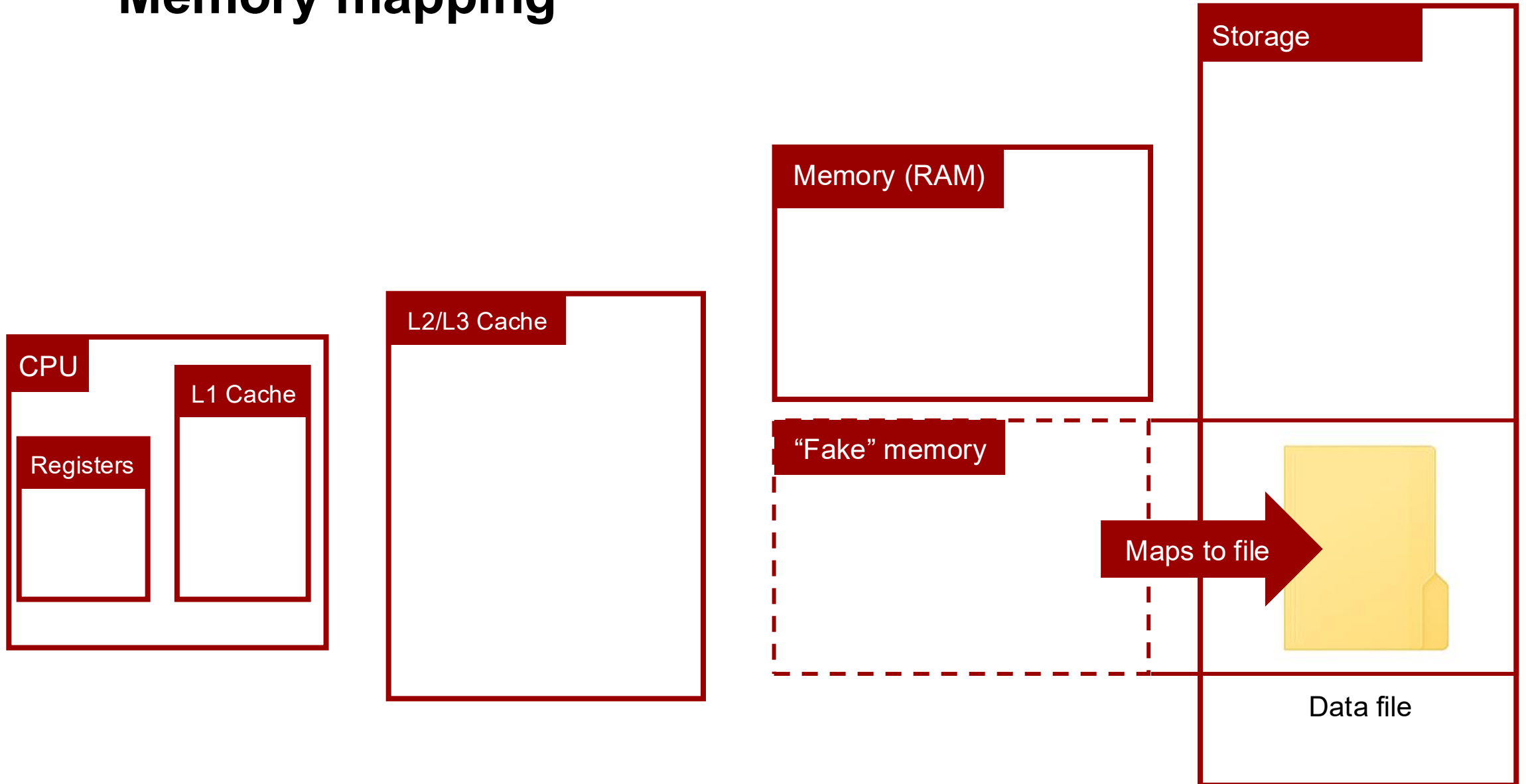
1. Load data from storage to memory
2. Load data from memory to caches
3. Do computation



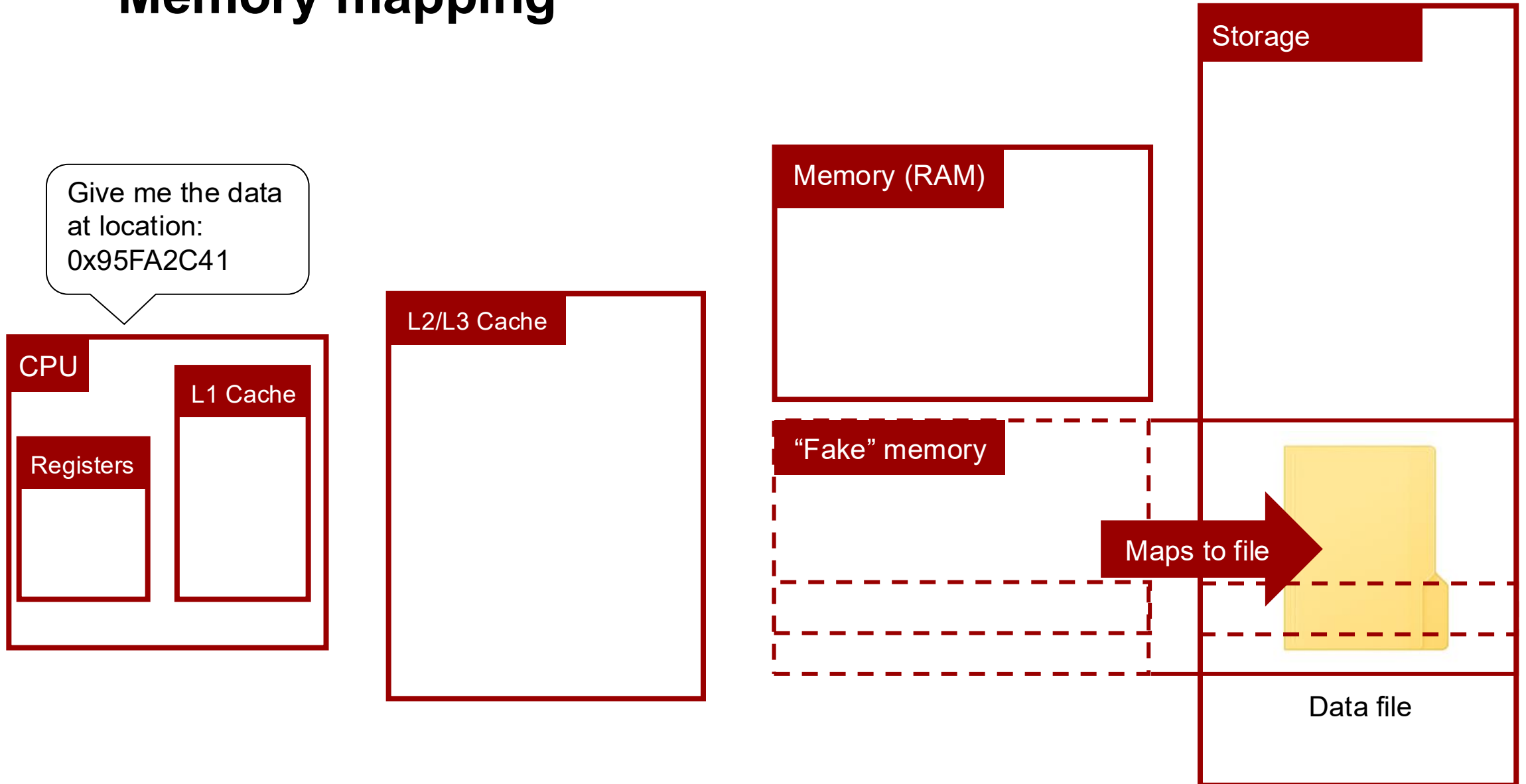
Memory mapping



Memory mapping



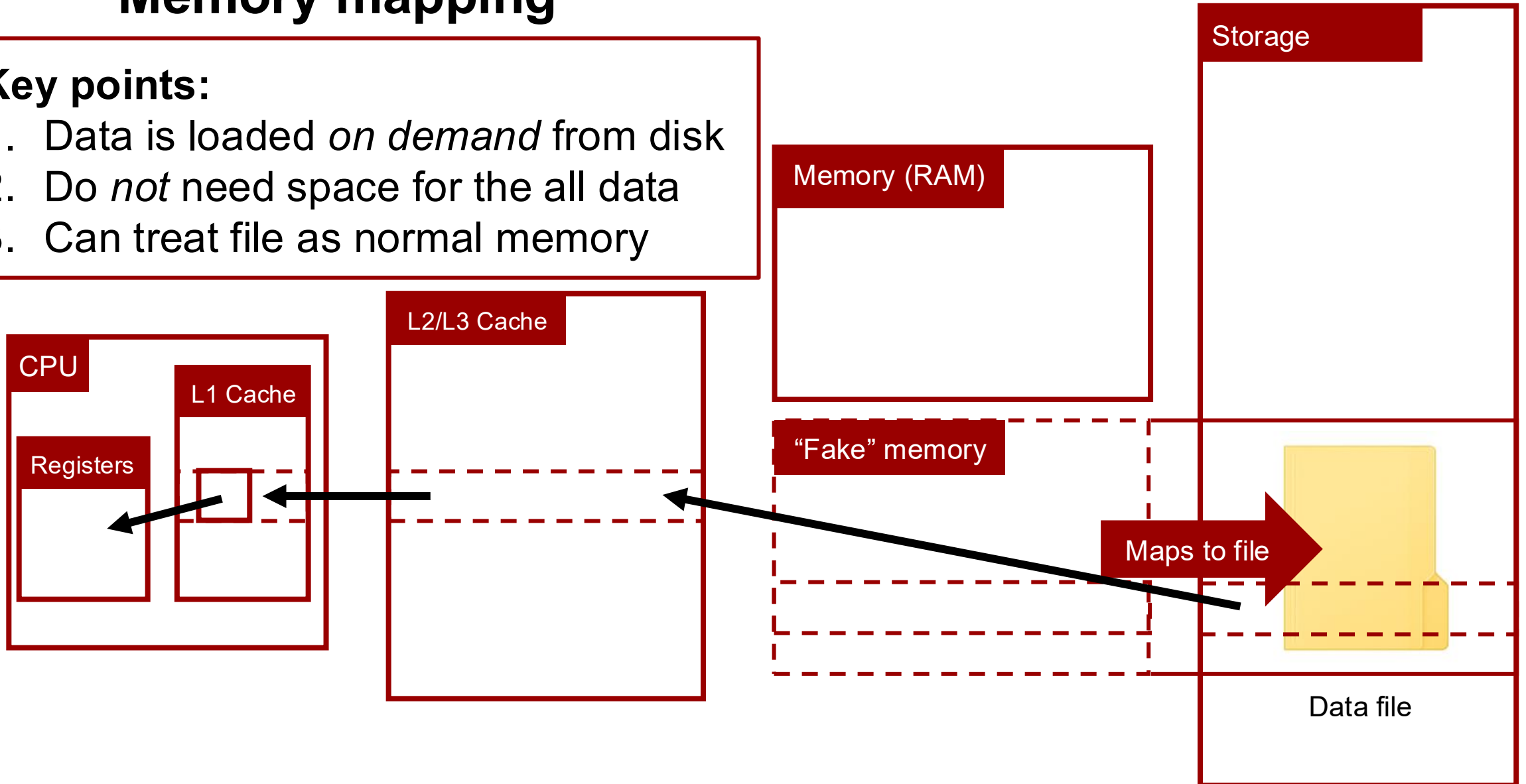
Memory mapping



Memory mapping

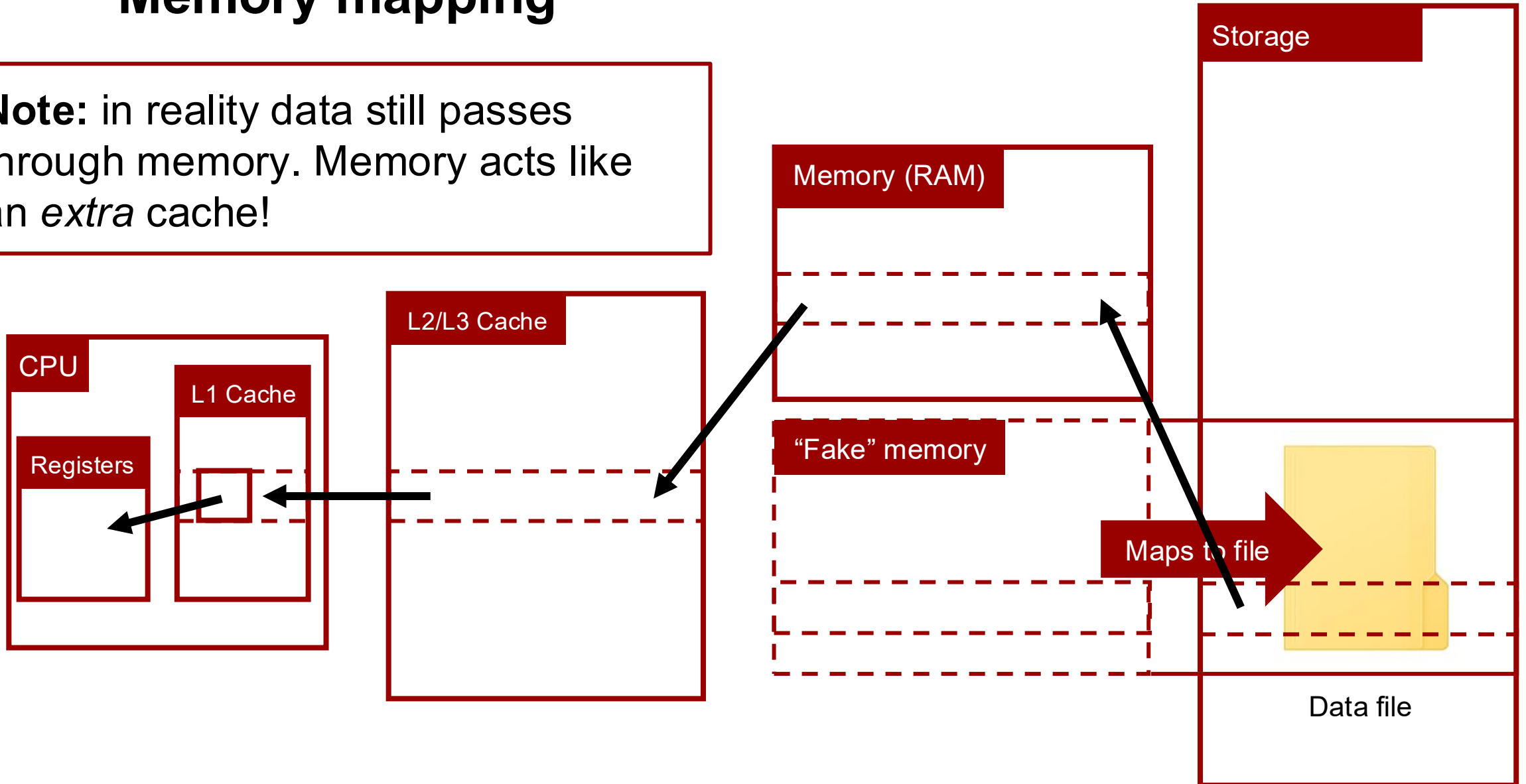
Key points:

1. Data is loaded *on demand* from disk
2. Do *not* need space for the all data
3. Can treat file as normal memory



Memory mapping

Note: in reality data still passes through memory. Memory acts like an *extra* cache!

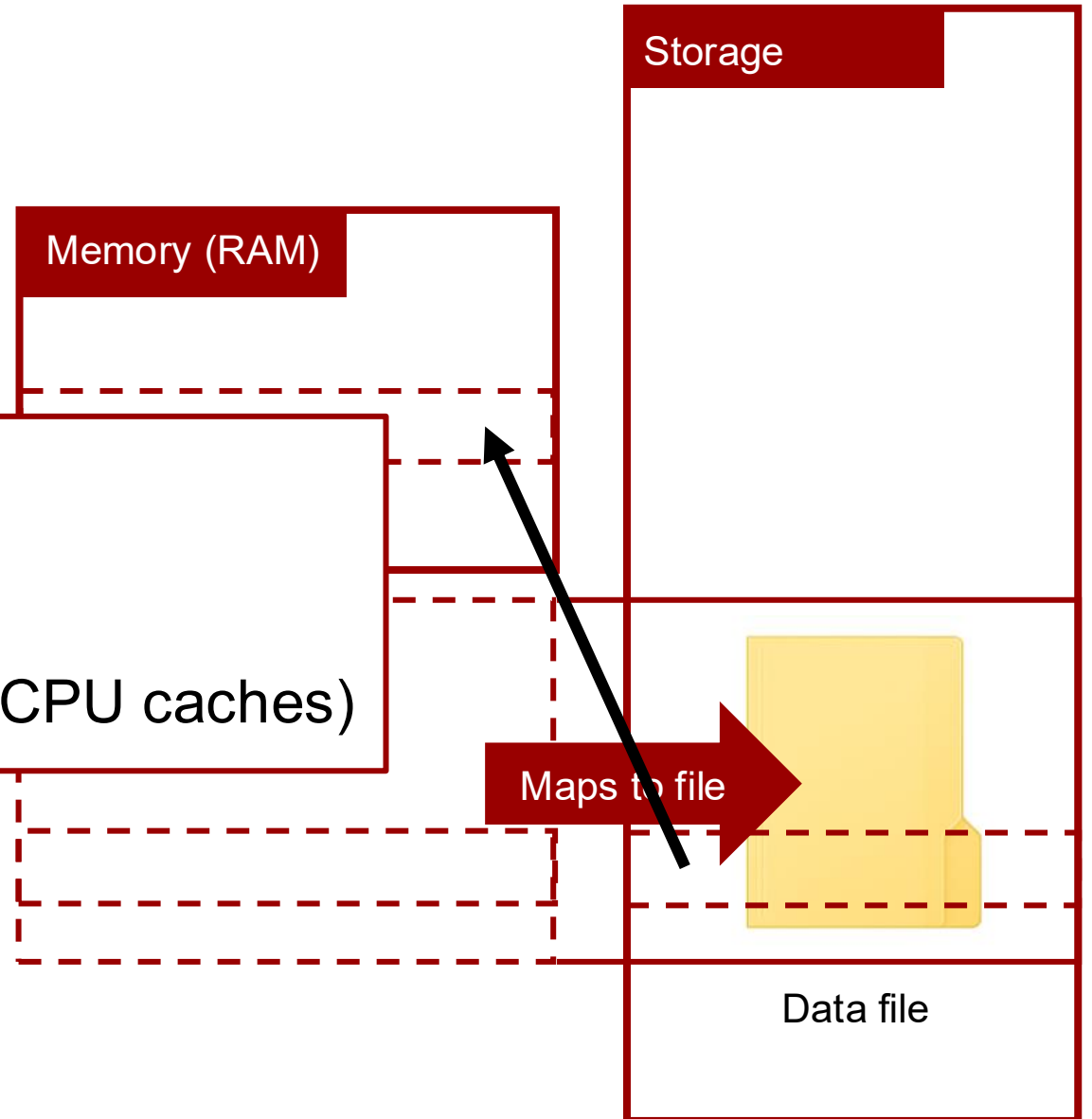


Memory mapping

Note: in reality data still passes through memory. Memory acts like an *extra* cache!

Consequences:

1. Reading / writing cached data = fast
2. Reading / writing from file = slow
3. Data evicted once memory is full (like CPU caches)



Memory mapping: NumPy

```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='w+', shape=(10, 10), dtype='float64')
```

File name

Mode

Array shape

Data type

'w+' = create / overwrite file

Memory mapping: NumPy

```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='w+', shape=(10, 10), dtype='float64')
>>> mm
memmap([[0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])
>>>
```

Memory mapping: NumPy

```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='w+', shape=(10, 10), dtype='float64')
>>> mm.dtype
dtype('float64')
>>> mm.shape
(10, 10)
```

Memory mapping: NumPy

```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='w+', shape=(10, 10), dtype='float64')
>>> mm[:5, :5] = 1
>>> mm
memmap([[1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])
>>>
```

Note: this changes the saved file too!

Memory mapping: NumPy

```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='r')
```

File name Mode

'r' = read only

Memory mapping: NumPy

```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='r')
>>> mm[0] = 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: assignment destination is read-only
```

Memory mapping: NumPy

```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='r+')
```

File name Mode

'r+' = read and write

Memory mapping: NumPy

```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='r+')
>>> mm[0] = 2
>>>
```


Memory mapping: NumPy

```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='r+')
>>> mm[0] = 2
>>> mm.shape
(800,)
>>>
```



???

Memory mapping: NumPy

```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='r+')
>>> mm[0] = 2
>>> mm.shape
(800,)
>>> mm.dtype
dtype('uint8')
>>> mm[:26]
```

```
memmap([ 2,  0,  0,  0,  0,  0, 240, 63,  0,  0,  0,  0,  0,
         0, 240, 63,  0,  0,  0,  0,  0,  0, 240, 63,  0,  0],
        dtype=uint8)
```

Except the 2!

Garbage data

Default dtype is uint8 – we are reading raw data!

float64 = 8 bytes

$10 * 10 * 8 = 800$

Memory mapping: NumPy

```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='r+')
>>> mm[0] = 2
>>> mm.shape
(800,)
>>> mm.dtype
dtype('uint8')
>>> mm[:].memmap('tmp.raw', mode='w+', dtype=uint8)
```

Default dtype is uint8 – we are reading raw data!

float64 = 8 bytes

Moral: must specify shape and data type for memmaps

Garbage data

Memory mapping: NumPy

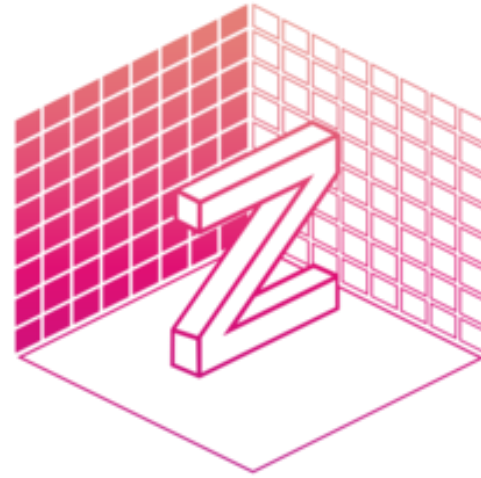
```
>>> import numpy as np
>>> mm = np.memmap('tmp.raw', mode='r+', dtype='float64', shape=(10, 10))
>>> mm
memmap([[1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])
```

Can now read and write
like normal NumPy array

Note: changes are written
out to disk! Be careful!

Quiz time!

New and shiny



Zarr

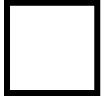


HDF5

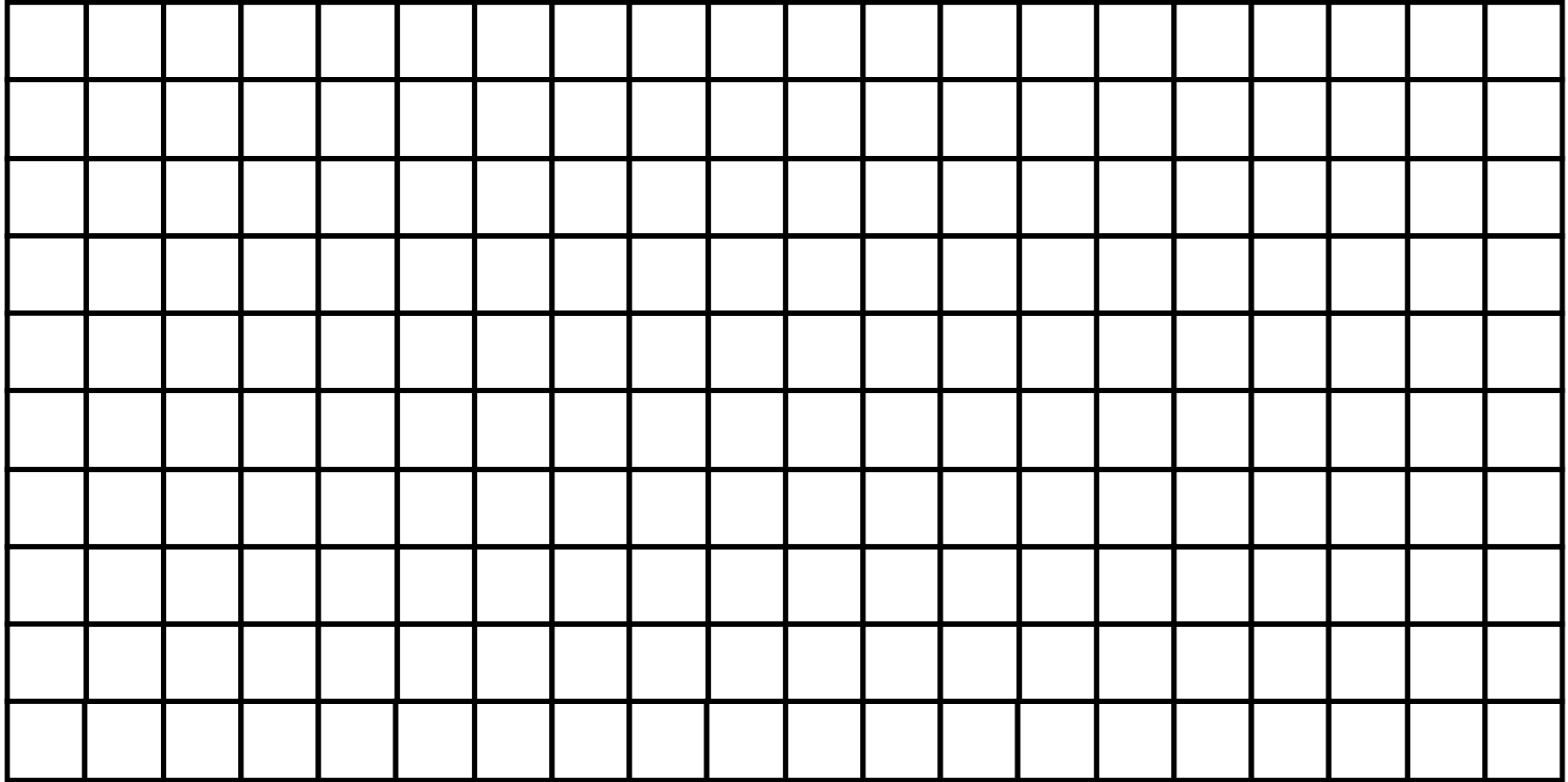
Old and established

Zarr is a file storage format for chunked, compressed, N-dimensional arrays based on an open-source specification.

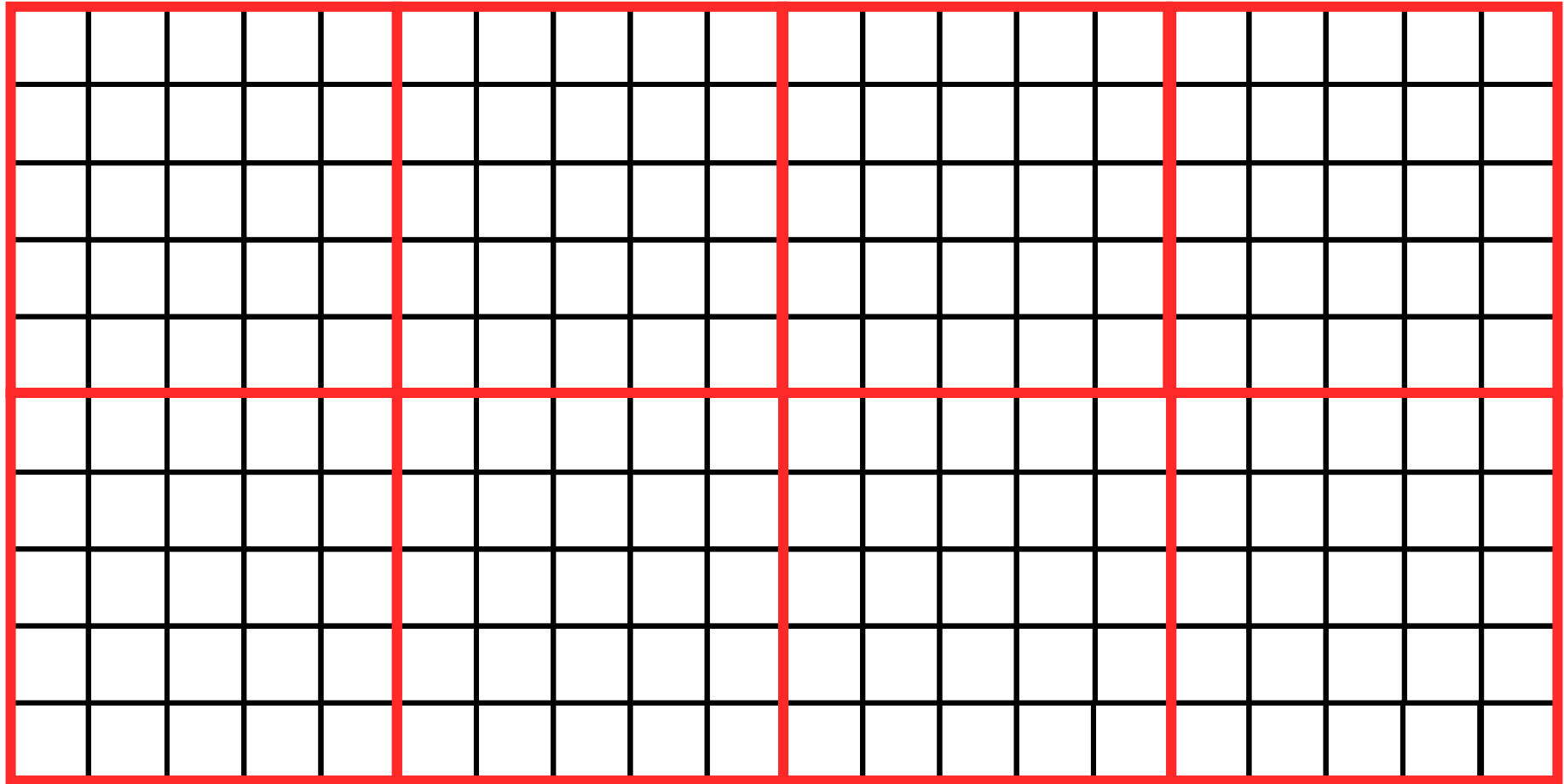
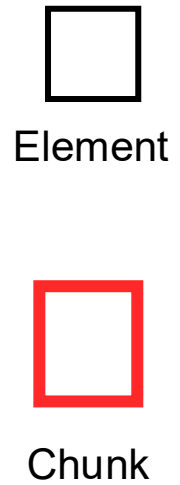
Chunked, compressed, N-dimensional arrays



Element



Chunked, compressed, N-dimensional arrays



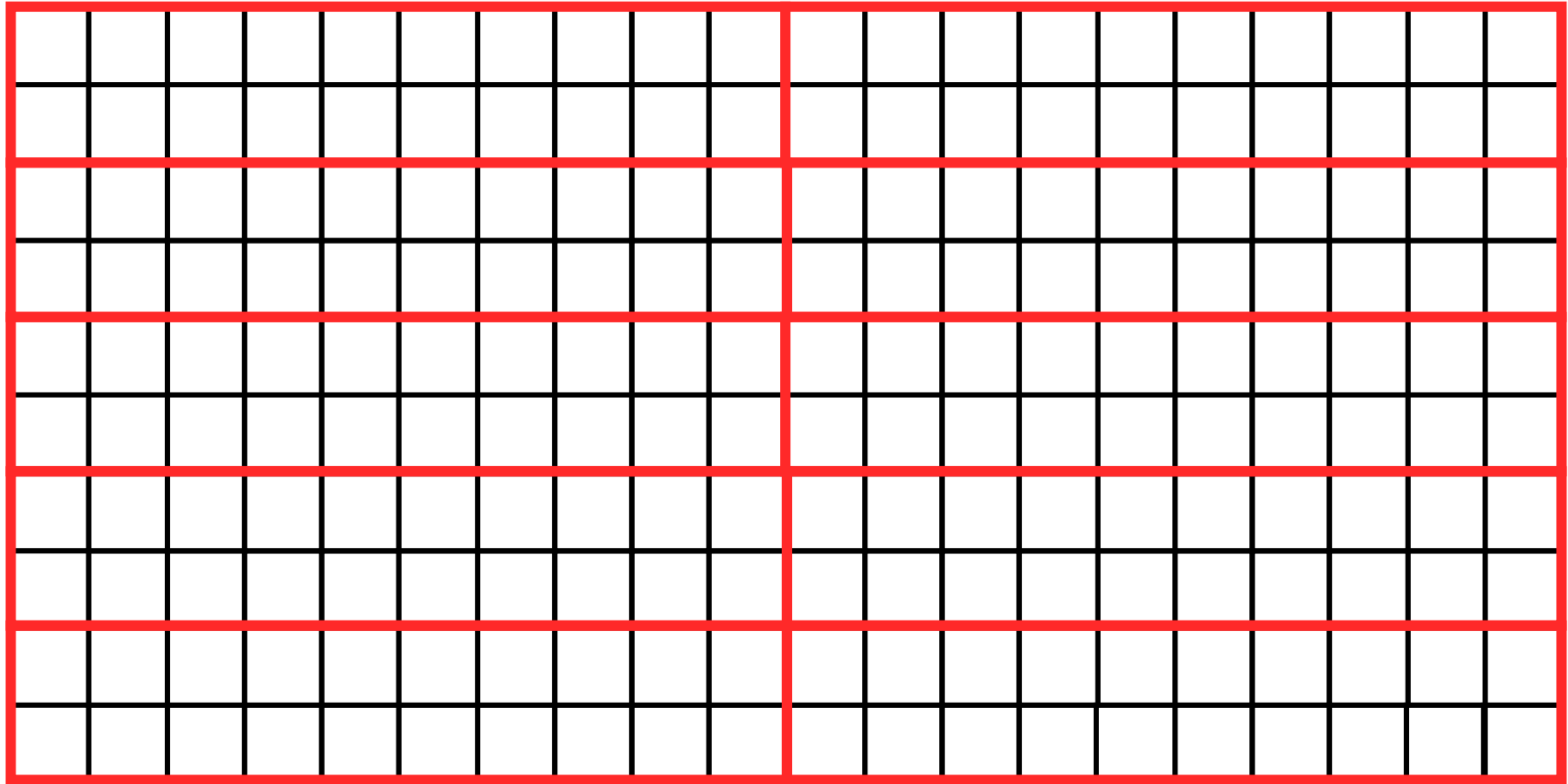
Chunked, compressed, N-dimensional arrays



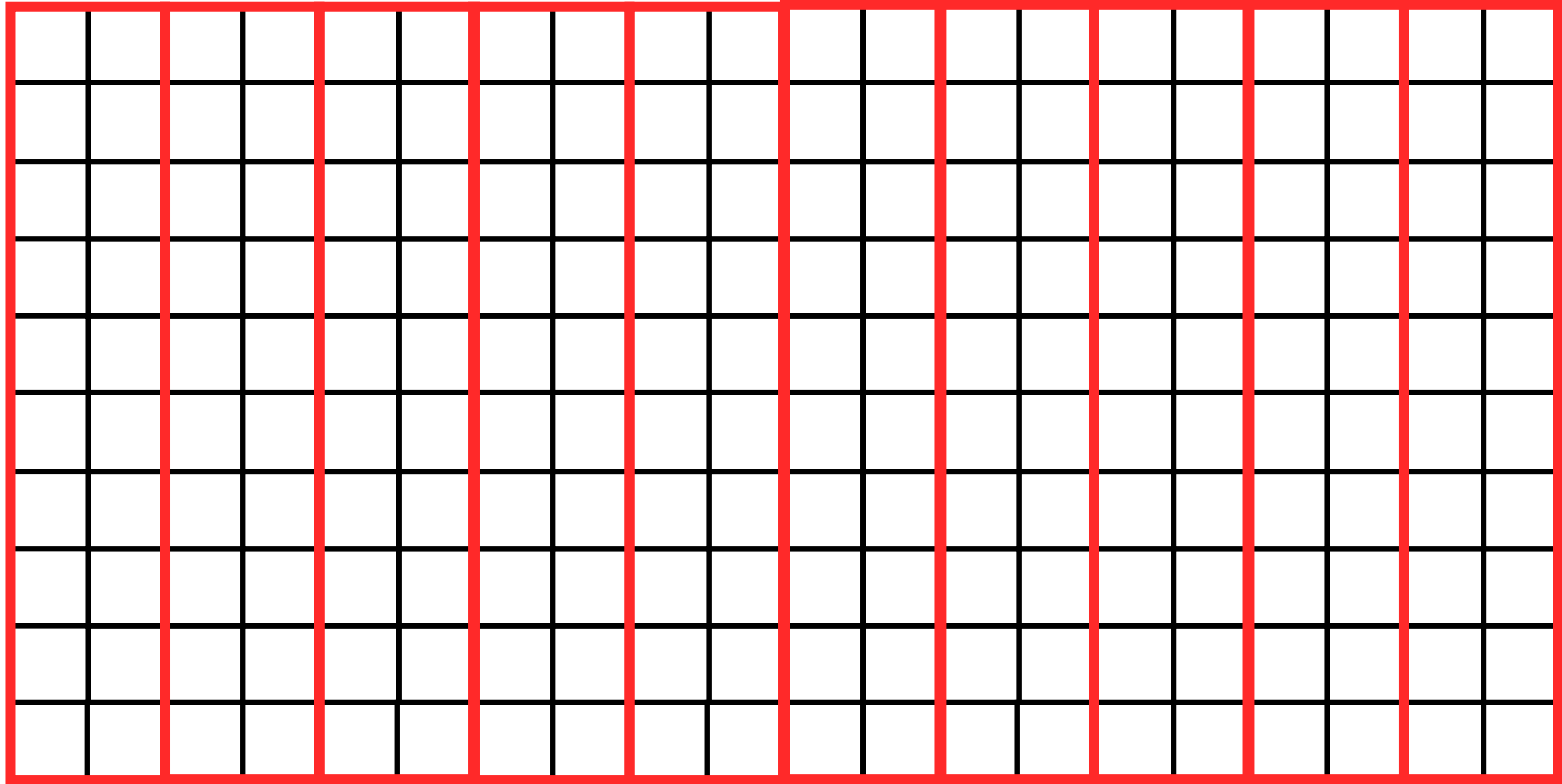
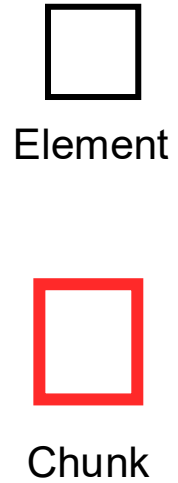
Element



Chunk



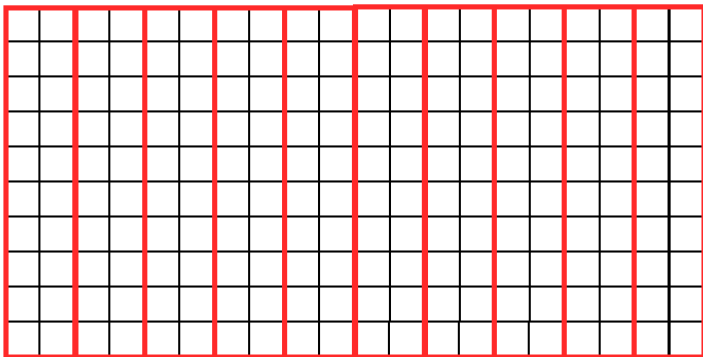
Chunked, compressed, N-dimensional arrays



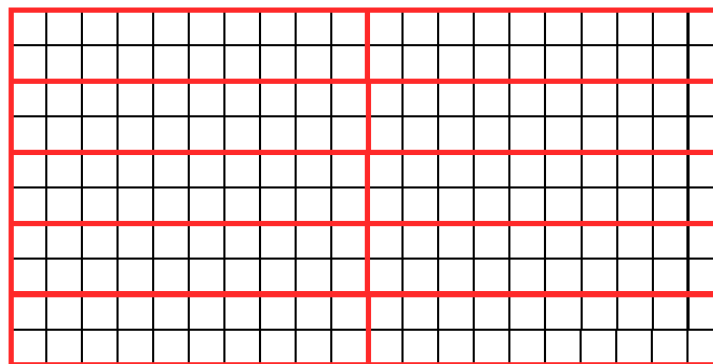
Chunked, compressed, N-dimensional arrays

Chunking allows us to tune the storage pattern to our access pattern

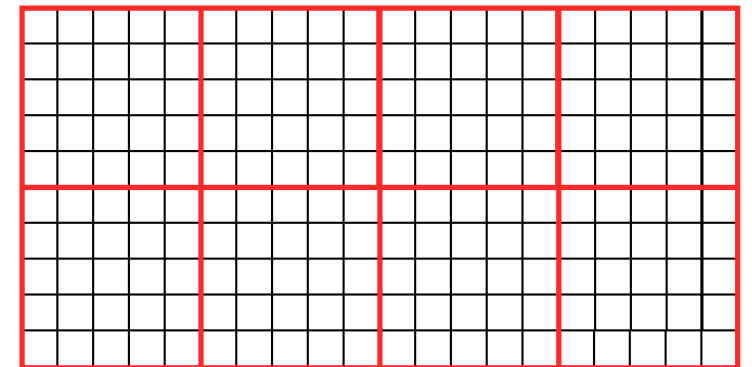
Good for “column-wise”



Good for “row-wise”



Good for “neighborhood-wise”



Chunked, compressed, N-dimensional arrays

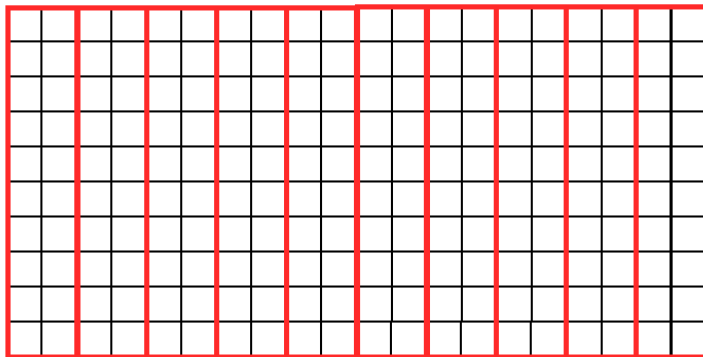
Chunking **forces** us to tune the storage pattern to our access pattern

Good performance depends on good chunk size and shape

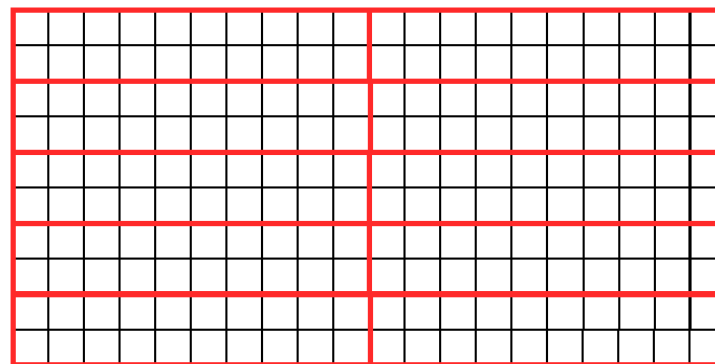
Should be at least 1 MB uncompressed

bit sized data (uint8) skulde man have 2^{20} elements in our chunk to have good performance, divided by how many bytes are in your datatype

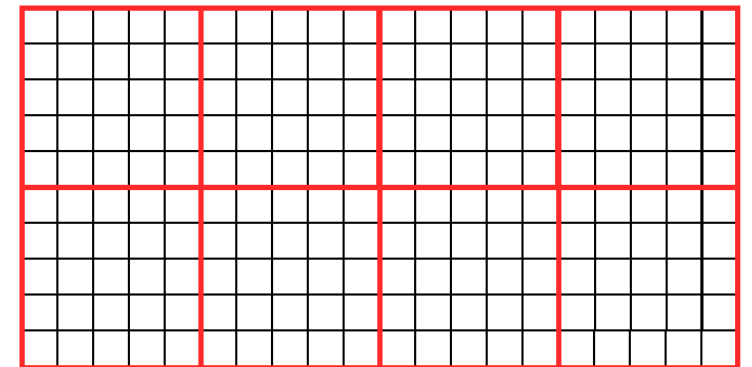
Good for “column-wise”



Good for “row-wise”



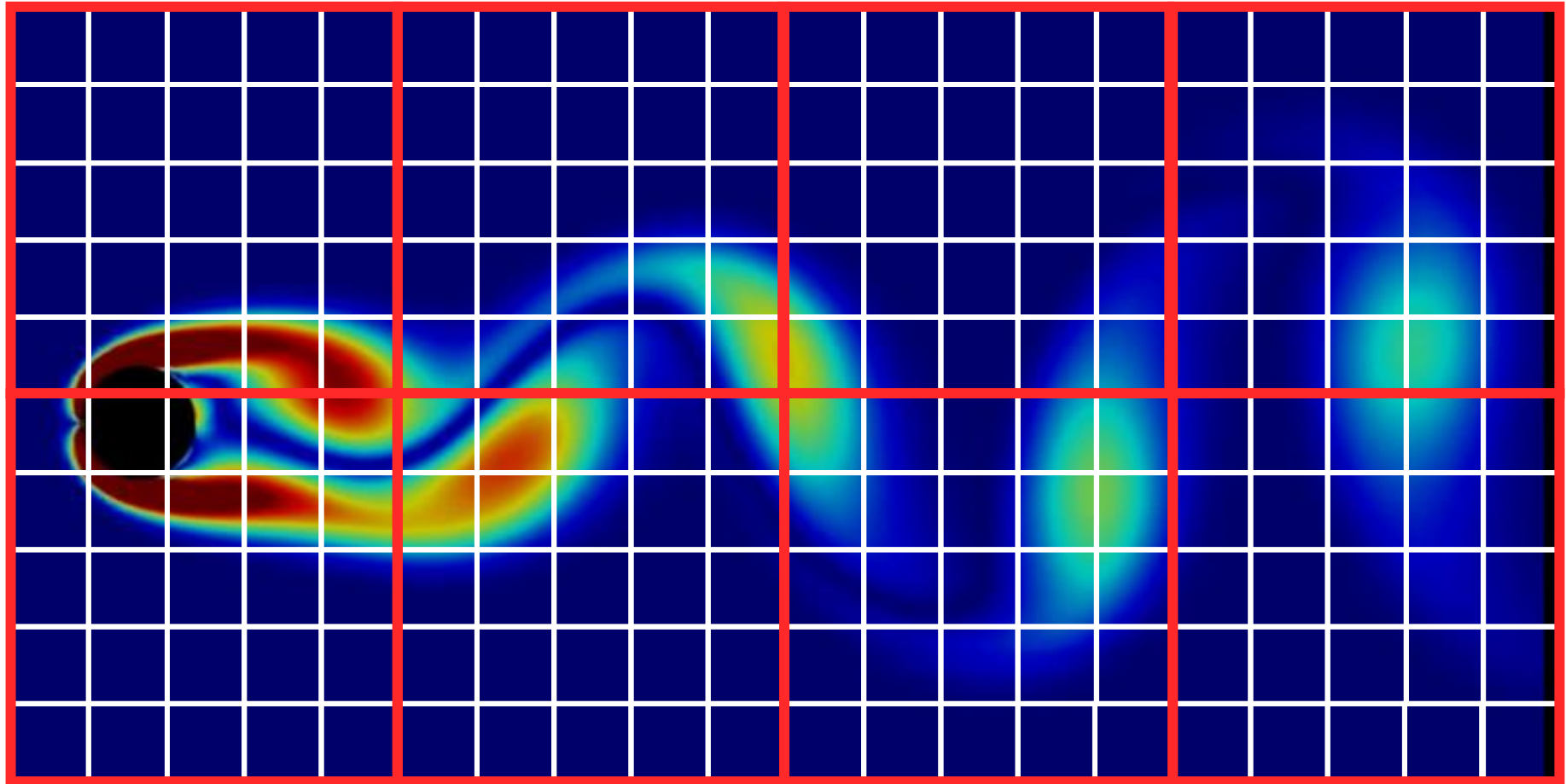
Good for “neighborhood-wise”



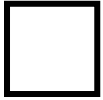
Chunked, compressed, N-dimensional arrays


Element


Chunk



Chunked, compressed, N-dimensional arrays

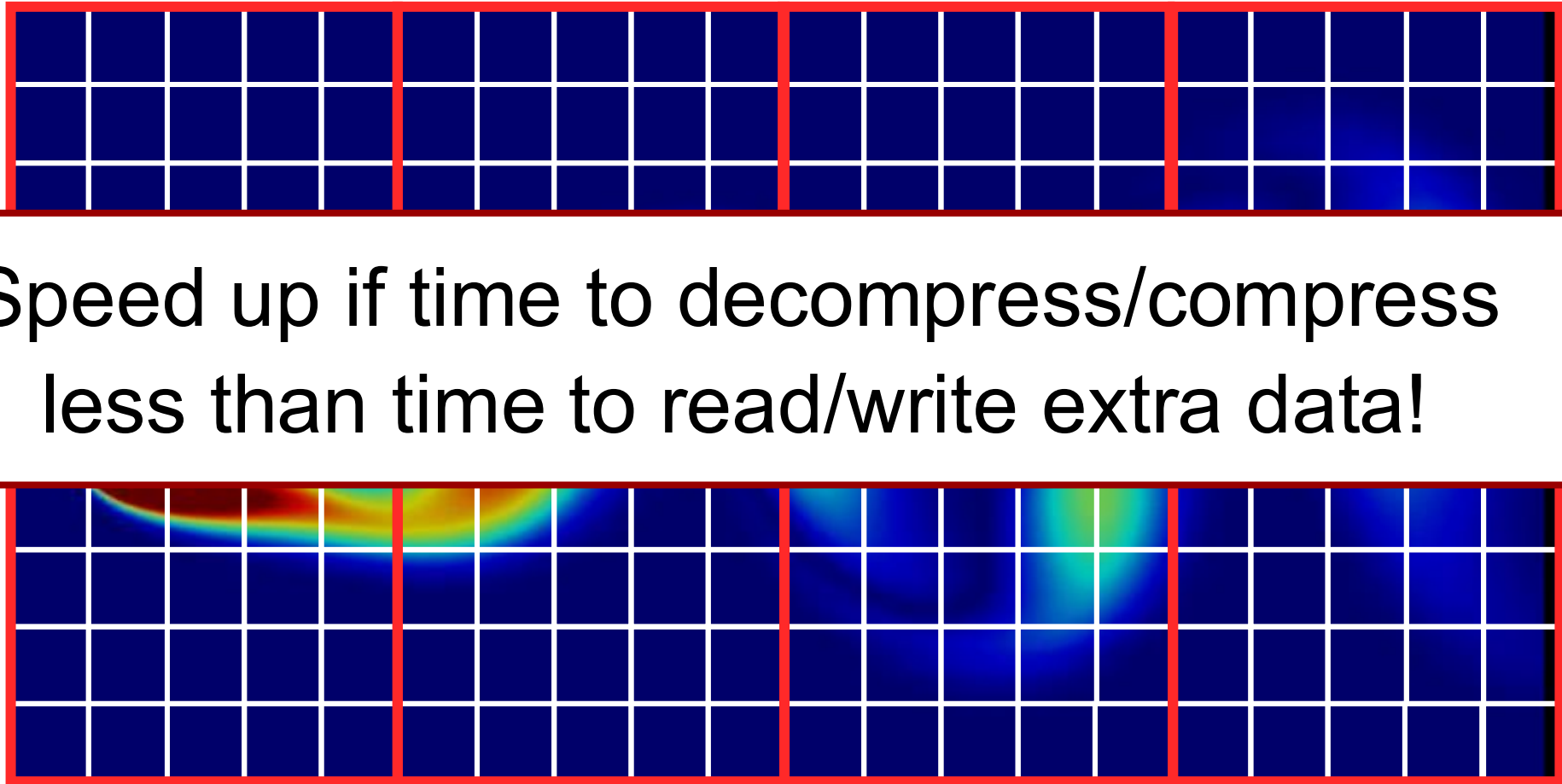


Element

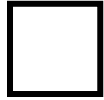


Chunk

Speed up if time to decompress/compress
less than time to read/write extra data!



Chunked, compressed, N-dimensional arrays

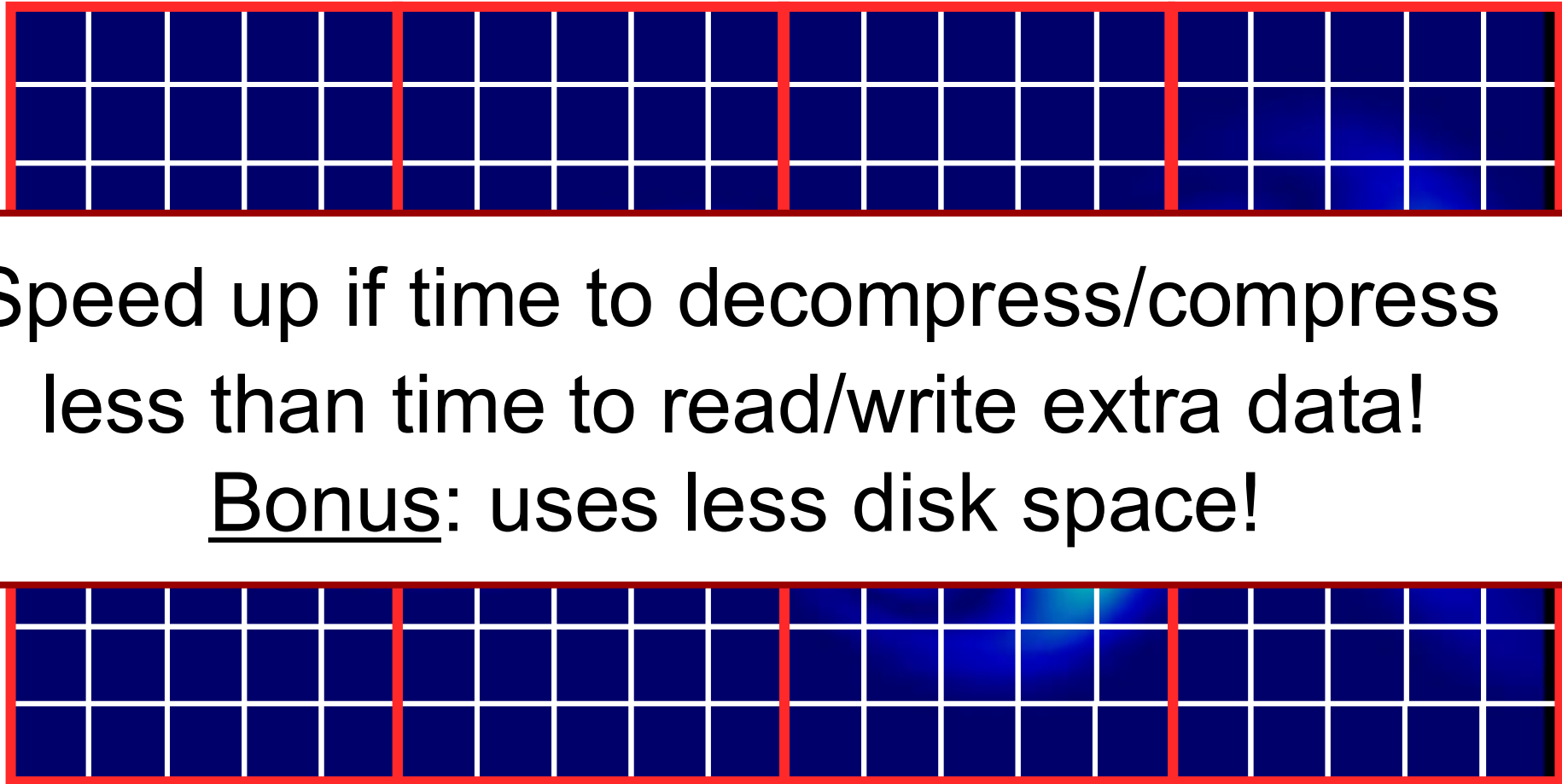


Element



Chunk

Speed up if time to decompress/compress
less than time to read/write extra data!
Bonus: uses less disk space!



Zarr

```
>>> import zarr  
>>> array = zarr.open('mandelbrot.zarr')
```


Zarr

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.info
Type                : zarr.core.Array
Data type           : int32
Shape               : (1000, 1000)
Chunk shape         : (200, 200)
Order               : C
Read-only           : False
Compressor          : Blosc(cname='lz4', clevel=5, shuffle=SHUFFLE, blocksize=0)
Store type          : zarr.storage.DirectoryStore
No. bytes           : 4000000 (3.8M)
No. bytes stored    : 95666 (93.4K)
Storage ratio       : 41.8
Chunks initialized  : 25/25
```



Zarr

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.info
Type                : zarr.core.Array
Data type           : int32
Shape               : (1000, 1000)   Shape of whole array
Chunk shape         : (200, 200)
Order               : C
Read-only           : False
Compressor          : Blosc(cname='lz4', clevel=5, shuffle=SHUFFLE, blocksize=0)
Store type          : zarr.storage.DirectoryStore
No. bytes           : 4000000 (3.8M)
No. bytes stored    : 95666 (93.4K)
Storage ratio       : 41.8
Chunks initialized  : 25/25
```

Zarr

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.info
Type                : zarr.core.Array
Data type           : int32
Shape               : (1000, 1000)
→ Chunk shape       : (200, 200)      Shape of each chunk
Order               : C
Read-only           : False
Compressor          : Blosc(cname='lz4', clevel=5, shuffle=SHUFFLE, blocksize=0)
Store type          : zarr.storage.DirectoryStore
No. bytes            : 4000000 (3.8M)
No. bytes stored    : 95666 (93.4K)
Storage ratio       : 41.8
Chunks initialized  : 25/25
```

Zarr

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.info
Type                : zarr.core.Array
Data type           : int32
Shape               : (1000, 1000)
Chunk shape         : (200, 200)
Order               : C
Read-only           : False
Compressor          : Blosc(cname='lz4', clevel=5, shuffle=SHUFFLE, blocksize=0)
Store type          : zarr.storage.DirectoryStore
No. bytes           : 4000000 (3.8M)
No. bytes stored    : 95666 (93.4K)
Storage ratio       : 41.8
Chunks initialized  : 25/25
```

→ Order : C Data order for each chunk.
C = row-wise, F = Fortran = column-wise

Zarr

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.info
Type                : zarr.core.Array
Data type           : int32
Shape               : (1000, 1000)
Chunk shape         : (200, 200)
Order               : C
Read-only           : False
Compressor          : Blosc(cname='lz4', clevel=5, shuffle=SHUFFLE, blocksize=0)
Store type          : zarr.storage.DirectoryStore
No. bytes           : 4000000 (3.8M)
No. bytes stored    : 95666 (93.4K)
Storage ratio       : 41.8
Chunks initialized  : 25/25
```

→

Chunk compression algorithm

Zarr

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.info
Type                : zarr.core.Array
Data type           : int32
Shape               : (1000, 1000)
Chunk shape         : (200, 200)
Order               : C
Read-only           : False
Compressor          : Blosc(cname='lz4', clevel=5, shuffle=SHUFFLE, blocksize=0)
Store type          : zarr.storage.DirectoryStore    Storage method
No. bytes           : 4000000 (3.8M)
No. bytes stored    : 95666 (93.4K)
Storage ratio       : 41.8
Chunks initialized  : 25/25
```



Zarr

...but not HDF5

```
>>> import zarr  
>>> array = zarr.open('mandelbrot.zarr')
```

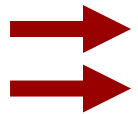
Zarr supports several storage backends, e.g.:

- DirectoryStore (standard for persistent storage)
- MemoryStore (everything in memory)
- SQLiteStore (backed by SQLite database)
- MongoDBStore (backed by MongoDB)
- Amazon S3, Google Cloud Storage, Azure Blob Storage
- LRUStoreCache (cache results from slower store)

```
Storage ratio      : 41.8  
Chunks initialized : 25/25
```

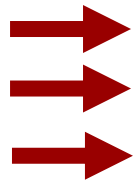
Zarr

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.info
Type                : zarr.core.Array
Data type           : int32
Shape               : (1000, 1000)
Chunk shape         : (200, 200)
Order               : C
Read-only           : False
Compressor          : Blosc(cname='lz4', clevel=5, shuffle=SHUFFLE, blocksize=0)
Store type          : zarr.storage.DirectoryStore
No. bytes            : 4000000 (3.8M)    Bytes needed to store full array
No. bytes stored     : 95666 (93.4K)    Bytes actually used after compression
Storage ratio        : 41.8
Chunks initialized  : 25/25
```



Zarr

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.info
Type                : zarr.core.Array
Data type           : int32
Shape               : (1000, 1000)
Chunk shape         : (200, 200)
Order               : C
Read-only           : False
Compressor          : Blosc(cname='lz4', clevel=5, shuffle=SHUFFLE, blocksize=0)
Store type          : zarr.storage.DirectoryStore
No. bytes           : 4000000 (3.8M)    Bytes needed to store full array
No. bytes stored    : 95666 (93.4K)    Bytes actually used after compression
Storage ratio       : 41.8             (No. bytes) / (No. bytes stored)
Chunks initialized : 25/25
```



Zarr

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.info
Type                : zarr.core.Array
Data type           : int32
Shape               : (1000, 1000)
Chunk shape         : (200, 200)
Order               : C
Read-only           : False
Compressor          : Blosc(cname='lz4', clevel=5, shuffle=SHUFFLE, blocksize=0)
Store type          : zarr.storage.DirectoryStore
No. bytes           : 4000000 (3.8M)
No. bytes stored    : 95666 (93.4K)
Storage ratio       : 41.8
Chunks initialized  : 25/25
```



Zarr

```
>>> import zarr
>>> empty = zarr.open('empty.zarr', mode='w', shape=(100,100), chunks=(20, 20))
```

File name

Mode

Array shape

Chunk shape

'w' = create file

Zarr

```
>>> import zarr
>>> empty = zarr.open('empty.zarr', mode='w', shape=(100,100), chunks=(20, 20))
>>> empty.info
```

Type	: zarr.core.Array
Data type	: float64
Shape	: (100, 100)
Chunk shape	: (20, 20)
Order	: C
Read-only	: False
Compressor	: Blosc(cname='lz4', clevel=5, shuffle=SHUFFLE, blocksize=0)
Store type	: zarr.storage.DirectoryStore
No. bytes	: 80000 (78.1K)
No. bytes stored	: 339
Storage ratio	: 236.0
Chunks initialized	: 0/25

Great for sparse data!

Only some metadata is actually stored!

No chunks initialized

Zarr: groups

```
>>> import zarr
>>> data = zarr.open('data.zarr', mode='w')
>>> data.create_group('group1')
>>> data.create_group('group2')
>>> data['group1'].create_group('arrays') # Nested group
>>> data.tree()
/
├── group1
│   └── arrays
└── group2
```

**Groups act like folders
in a file system**
(also available in HDF5)

Zarr: create array in group

```
>>> import zarr
>>> data = zarr.open('data.zarr', mode='w')
>>> data.create_group('group1')
>>> data.create_group('group2')
>>> data['group1'].create_group('arrays') # Nested group
>>> data['group1/arrays'].zeros('array1', shape=(100, 100), chunks=(50, 20))
```

Array "path"

Array name

Array shape

Chunk shape

Zarr: assign array to group

```
>>> import zarr
>>> data = zarr.open('data.zarr', mode='w')
>>> data.create_group('group1')
>>> data.create_group('group2')
>>> data['group1'].create_group('arrays') # Nested group
>>> data['group1/arrays'].zeros('array1', shape=(100, 100), chunks=(50, 20))
>>> data['group1/arrays/array2'] = zarr.zeros(shape=(100, 100), chunks=(50, 20))
```

Array "path"

Array name

Array shape

Chunk shape

Zarr: list contents

```
>>> import zarr
>>> data = zarr.open('data.zarr', mode='w')
>>> data.create_group('group1')
>>> data.create_group('group2')
>>> data['group1'].create_group('arrays') # Nested group
>>> data['group1/arrays'].zeros('array1', shape=(100, 100), chunks=(50, 20))
>>> data['group1/arrays/array2'] = zarr.zeros(shape=(100, 100), chunks=(50, 20))
>>> data.tree()
/
├── group1
│   └── arrays
│       ├── array1 (100, 100) float64
│       └── array2 (100, 100) float64
└── group2
```

Zarr works as dataset storage, not just arrays

Zarr: internal format

```
$ ls  
test.zarr  
$ ls test.zarr  
group1  group2
```

Zarr: internal format

```
$ ls
test.zarr
$ tree test.zarr
test.zarr/
├── group1
│   └── arrays
│       ├── array1
│       └── array2
│           ├── 0.0
│           ├── 0.1
│           ├── ...
│           ├── 1.2
│           ├── 1.3
│           └── 1.4
└── group2
```

Chunks

Zarr (with DirectoryStore) stores data in an *actual* directory (/folder)
Groups and arrays are sub-folders
Chunks are separate files

Zarr: internal format

```
$ ls
test.zarr
$ ls -a test.zarr
.  ..  group1  group2  .zgroup
$ ls -a test.zarr/group1/arrays/array1
.  ..  .zarray
```

Zarr (with DirectoryStore) stores data in an *actual* directory (/folder)

Groups and arrays are sub-folders

Chunks are separate files

Meta data are in files

Zarr: internal format

```
$ ls
test.zarr
$ ls -a test.zarr
.  ..  group1  group2  .zgroup
$ ls -a test.zarr/group1/arrays/array1
.  ..  .zarray
```

Consequence:

Can read *and* write to
chunks in parallel!
(write not possible in HDF5)

**Zarr (with DirectoryStore) stores
data in an *actual* directory (/folder)**

Groups and arrays are sub-folders

Chunks are separate files

Meta data are in files

Zarr: indexing

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array[:5, :10]
array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]], dtype=
>>> type(array[:5, :10])
<class 'numpy.ndarray'>
```

Can index like NumPy array and get a NumPy array(!) out.

- Python indexing: [start:end:step]
- Boolean indexing: [True, True, ...]
- Array of indexes: [1, 2, 3]

Note: be careful for large arrays!

Zarr: block indexing

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.shape
(1000, 1000)           Shape of whole array
>>> array.chunks
(200, 200)            Shape of each chunk
>>> array.cdata_shape
(5, 5)                Chunks per dimension
```

Zarr: block indexing

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.blocks[0, 1]
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [1, 1, 1, ..., 2, 2, 2],
       [1, 1, 1, ..., 2, 2, 2],
       [1, 1, 1, ..., 2, 2, 2]], dtype=int32)
>>> array.blocks[0, 1].shape
(200, 200)
>>> array.chunks
(200, 200)
```

Zarr: block indexing

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.blocks[0, 1]
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [1, 1, 1, ..., 2, 2, 2],
       [1, 1, 1, ..., 2, 2, 2],
       [1, 1, 1, ..., 2, 2, 2]], dtype=int32)
>>> (array.blocks[0, 1] == array[:200, 200:400]).all()
True
```


Zarr: block indexing

```
>>> import zarr
>>> array = zarr.open('mandelbrot.zarr')
>>> array.blocks[0, 1] = 8    # Could also assign array instead of single number
>>> array.blocks[0, 1]
array([[8, 8, 8, ..., 8, 8, 8],
       [8, 8, 8, ..., 8, 8, 8],
       [8, 8, 8, ..., 8, 8, 8],
       ...,
       [8, 8, 8, ..., 8, 8, 8],
       [8, 8, 8, ..., 8, 8, 8],
       [8, 8, 8, ..., 8, 8, 8]], dtype=int32)
```

Quiz time!

Today's exercise

Computing in memory constrained environments

- Pandas with chunked reading
- Mandelbrot with NumPy memory maps
- Mandelbrot with Zarr arrays

Useful concepts

Pandas

```
df = pd.read_csv('data.csv', chunksize=C)
```

NumPy memmap

```
mode = 'w+'      # Make new file
mode = 'r+'      # Read and write
mode = 'r'       # Read only
mm = np.memmap('file.raw', mode=mode,
               shape=(N,M), dtype='int32')
```

Zarr

```
array = zarr.open('mandelbrot.zarr')
array.shape          # Array shape
array.chunks         # Chunk shape
array.cdata_shape    # Chunks per dimension
array.blocks[0, 1]   # Chunk access
```

Change to work node

```
linuxsh
```

Submit job script

```
bsub < submit.sh
```

Job status

```
bstat / bjobs
```

Check job output

```
bpeek / bpeek <JOBID>
```

Kill job

```
bkill <JOBID>
```